

أنواع البيانات (Data Types) في بايثون: الأعداد والنصوص

Quick Recap

- **Data Types (أنواع البيانات):** في عالم البرمجة، كل قيمة لها "نوع" يخبر الحاسوب بكيفية التعامل معها. الأنواع الأساسية الأكثر شيوعًا هي الأعداد والنصوص.
- **Numbers (الأعداد):** تميز لغة بايثون بين نوعين رئيسيين من الأعداد:
 - **Integers (int):** الأعداد الصحيحة التي لا تحتوي على فاصلة عشرية (مثل 10، -5، 100).
 - **Floats (float):** الأعداد التي تحتوي على فاصلة عشرية (مثل 10.5، -0.7، 99.0).
- **Strings (str):** البيانات النصية، والتي يجب أن تكون محاطة بعلامتي اقتباس، إما مفردة (' ') أو مزدوجة (" "). (مثل " Hello"، 'Python'، "123")
- **TypeError:** يحدث هذا الخطأ عند محاولة إجراء عملية بين أنواع بيانات غير متوافقة، مثل جمع عدد مع نص.
- **String Concatenation (دمج النصوص):** استخدام عامل + بين نصين يقوم بدمجهما معًا لإنشاء نص واحد. على سبيل المثال، 'Hello' + 'World' ينتج عنها 'HelloWorld'.

In-depth Explanation

مقدمة حول أنواع البيانات والخطأ TypeError (00:00 - 01:23)

في البرمجة، يُعد نوع البيانات التي تتعامل معها أمرًا بالغ الأهمية. تحتاج بايثون إلى معرفة ما إذا كانت القيمة عددًا (يمكن استخدامه في العمليات الحسابية) أم نصًا (تُعامل كتسلسل من الرموز) - سيتم دراسة هذا الجزء في القسم التاسع - .

عندما تخطئ بين أنواع غير متوافقة، تطلق بايثون خطأً يسمى **TypeError**. على سبيل المثال، لا يمكنك جمع عدد رياضيًا مع سلسلة نصية.

- **مثال من الفيديو:** يحاول هذا الرمز جمع العدد 5 مع النص "5".

```
# This will cause an error
print(5 + '5')
```

النتائج:

```
'TypeError: unsupported operand type(s) for +: 'int' and 'str'
```

تعني رسالة الخطأ هذه أنه لا يمكنك استخدام عامل `+` بين عدد صحيح (`int`) وسلسلة نصية (`str`).
• مثال إضافي: بالطريقة ذاتها، لا يمكن جمع عدد مع كلمة.

```
# This will also cause an error
print(10 + "hello")
```

ثلاثة أنواع أساسية للبيانات في بايثون (06:21 - 01:23)

فيما يلي أنواع البيانات الأساسية التي استعرضناها حتى الآن:

1. Integers (`int`)

هي الأعداد الصحيحة، سواء كانت موجبة أم سالبة، دون أي جزء عشري.

```
# An integer
print(100)

# A negative integer
print(-50)
```

2. Floats (`float`)

هي الأعداد التي تحتوي على فاصلة عشرية. نعرف أيضًا بأعداد الفاصلة العائمة.

```
# A float number
print(5.0)

# Another float
print(1230.75)
```

```
# A negative float
print(-9.81)
```

3. Strings (str)

السلسلة النصية هي تتابع من الرموز المحاطة بعلامتي اقتباس، إما مفردة (' ') أو مزدوجة (" "). يمكن أن تكون حرفاً، أو كلمة، أو جملة، أو حتى أعداداً تُعامل كنص.

```
# A string using double quotes
print("codezilla")

# A string using single quotes
print('islam')

# A string can contain numbers and spaces
print("my name is islam 40")

# A string can be a single character
print("a")
```

العمليات على أنواع البيانات: دمج النصوص (Concatenation)

(06:21 - 10:14)

بينما يقوم عامل + بإجراء عملية الجمع للأعداد (int و float)، يتغير سلوكه مع النصوص. فهو يقوم بعملية الدمج (concatenation)، أي أنه يربط السلاسل النصية ببعضها. هذا يشبه كتلة join في سكر إتش.

- مثال من الفيديو: دمج سلسلتين نصيتين. لاحظ عدم وجود مسافة بينهما في الناتج.

```
# The output will be "53", not 8
print("5" + "3")
```

الناتج:

53

- كيفية إضافة مسافة: لإضافة مسافة، يجب تضمينها كسلسلة نصية مستقلة.

```
# Method 1: Adding a space string in the middle
print("5" + " " + "3")

# Method 2: Adding a space inside one of the strings
print("5 " + "3")
```

النتائج (كلا الطريقتين):

3 5

• مثال إضافي: دمج كلمات لتكوين جملة.

```
print("!" + "مرحبًا" + " " + "أيها العالم")
```

النتائج:

مرحبًا أيها العالم!

Further Learning

• التوثيق الرسمي لبايثون: للحصول على نظرة شاملة، اقرأ الدليل الرسمي حول [Python's built-in types](#).

• مفاهيم أساسية للبحث عنها:

• **Type Casting (تحويل الأنواع)**: عملية تحويل نوع بيانات إلى آخر (مثل استخدام `str()` للتحويل إلى نص، أو `int()` للتحويل إلى عدد صحيح).

• **"لغات قوية التتميط" (Strongly Typed) مقابل "لغات ضعيفة التتميط" (Weakly Typed)**: ابحث عن هذا المصطلح لفهم سبب كون بايثون صارمة بشأن توافق الأنواع مقارنة بلغات أخرى مثل `JavaScript`.